

DVWA Penetration Test

by M-Security

Start of testing: July 10, 2026

End of testing: August 12, 2026

Contents

1 Executive Summary **1**

2 Vulnerability overview **3**

3 Results **4**

3.1 DVWA Web Application (DVWA) 4

3.1.1 Reflected XSS 4

3.1.1.1 Minimal proof of concept 5

3.1.1.2 Proposed solutions 5

3.1.2 Command Execution (Injection) 7

3.1.2.1 Minimal proof of concept 8

3.1.2.2 Proposed solutions 8

3.1.3 File Upload 10

3.1.3.1 Minimal proof of concept 10

3.1.3.2 Proposed solutions 12

3.1.4 File Inclusion 13

3.1.4.1 Minimal proof of concept 13

3.1.4.2 Proposed solutions 16

3.1.5 SQL Injection Without Randomization 17

3.1.5.1 Minimal proof of concept 17

3.1.5.2 Proposed solutions 18

3.1.6 Brute-Force Login 19

3.1.6.1 Minimal proof of concept 19

3.1.6.2 Proposed solutions 21

4 Appendices **23**

4.1 Network Scan Results 23

4.2 Reference and tool links 23

1 Executive Summary

In this penetration test, the Damn Vulnerable Web Application (DVWA) was examined for security-relevant weaknesses. The assessment used a gray-box approach, meaning that some information about the target environment was provided beforehand. This included information about the network range and guidance for performing the assigned penetration-testing exercises. The assessment included both web application testing and network reconnaissance.

The scope of the assessment was as follows:

- DVWA Web Application
  - Server IP address: 193.167.189.112
  - DVWA services were tested through the assigned ports.
- Network: 173.3.106.0/29
  - The network was scanned to identify active hosts, open ports and running services.

Table 1.1 contains the overview of examined systems during the penetration test.

Asset	**Address**
DVWA Web Application	193.167.189.112:[DVWA_PORT]
Network	173.3.106.0/29

Table 1.1: Systems examined during the penetration test

Several vulnerabilities were identified during the assessment. The identified web application vulnerabilities included reflected cross-site scripting (XSS), command injection, file upload, file inclusion, SQL injection and a brute-force login vulnerability. The network scan was additionally used to identify active hosts, open ports and services within the specified network range.

The vulnerabilities identified can have different levels of impact. The most significant findings included command injection, file upload and brute-force vulnerability, which could potentially allow an attacker to access or modify files on the underlying host. The reflected XSS vulnerability could allow malicious JavaScript to be executed in a victim's browser, while the SQL injection vulnerability could allow unauthorized access to information stored in the database. The file inclusion vulnerability allowed files to be read through the web application.

The remediation effort varies between the identified issues. Short-term solutions should focus on reducing the immediate attack surface by restricting file uploads, limiting authentication attempts and preventing user input from being executed as commands or SQL queries. Long-term solutions should include secure coding practices, input validation, secure file handling and stronger authentication controls.

Detailed descriptions of the vulnerabilities, their CVSS scores, business impacts, proof-of-concept steps and proposed remediation measures are provided in the Results chapter.

2 Vulnerability overview

Table 2.1 depicts all vulnerabilities found during the penetration test. They are categorized by their risk and potential and are differentiated in the categories low, medium, high and critical.

Low vulnerabilities have a limited security impact and are generally difficult to exploit or require additional conditions. Medium vulnerabilities may provide some unauthorized access or information but have a more limited overall impact. High vulnerabilities are relatively easy to exploit and can significantly affect the confidentiality, integrity, or availability of the system. Critical vulnerabilities represent the most serious findings and may allow an attacker to gain extensive control over the application or underlying system.

Figure 2.1 shows the overview of vulnerabilities grouped by target.

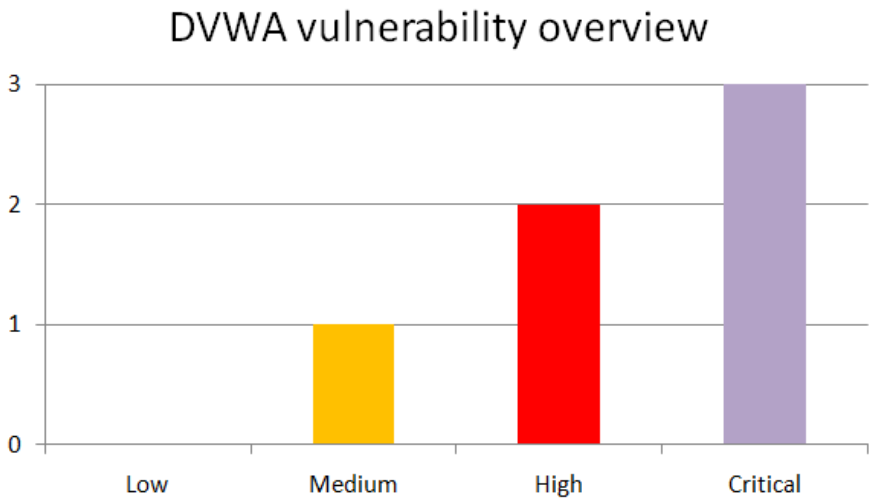


Figure 2.1: Vulnerability overview

Risk	Asset	Vulnerability	Section	Page
High	DVWA	Reflected XSS	3.1.1	4
Critical	DVWA	Command Execution (Injection)	3.1.2	7
Critical	DVWA	File Upload	3.1.3	10
Medium	DVWA	File Inclusion	3.1.4	13
High	DVWA	SQL Injection Without Randomization	3.1.5	17
Critical	DVWA	Brute-Force Login	3.1.6	19

Table 2.1: Vulnerability overview

3 Results

In this chapter, the vulnerabilities found during the penetration test are presented. All the issues contain the following information:

- Brief description of the vulnerability.
- CVSS Base Score – see here for details.
- Exploitability – describes the likelihood of an issue being used against customer's infrastructure.
- Business impact.
- References to classifications: OWASP, CWE.
- Steps to reproduce the attack and exploit the vulnerability.

Also the remediation recommendations are given for each issue found during the penetration test. Both short and long term solutions are presented.

3.1 DVWA Web Application (DVWA)

The targeted system is the Damn Vulnerable Web Application (DVWA) instance that is hosted on the following server IP address. The application was investigated through its exposed HTTP service. The server was using Apache Tomcat with the Coyote JSP engine 1.1.
Hostname: `http//:193.167.189.112`
Server IP address: 193.167.189.112:<DVWA_PORT>

3.1.1 Reflected XSS

A reflected cross site scripting (XSS) attack means that a user sends an input to the targeted web application that does not check the validity of the input which leads to the web application resending the data that the user wants back to the user. In this case, we wrote a script described in the minimal proof of concept, and sent it to an input box in the DVWA's "XSS (Reflected)" website. Right after, DVWA's backend took the script and executed it, causing the site to give us an alert.

Basic information about this issue is presented in Table 3.1.

Description	A simple JavaScript payload that recovers normally unauthorized data.
CVSS Base Score	8.2
Exploitability	High
Business impact	A threat actor could potentially get sensitive information about the business or a separate victim. Additionally, the attacker could manipulate the content displayed to the victim.
References to classifications	OWASP
	CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)
Affected input	The user-controlled input field in the DVWA page where the XSS payload was submitted.
Affected output	• A JavaScript alert dialog was triggered. • The alert displayed the message `Emergency123`.

Table 3.1: Issue #1: Reflected XSS vulnerability on the DVWA website

3.1.1.1 Minimal proof of concept

When entering the DVWA reflected cross site scripting (XSS) subsite, we see an input field where the user can input anything, even JavaScript as shown in Figure 3.1.

When we tried writing with regular JavaScript syntax, we saw that the website had changed the special characters around, meaning there is some output encoding in place. That is why we had to try a few times before we understood which special characters are converted into the actual JavaScript characters we need for the alert to show up. We came up with this payload:

```
}script$alert>'Emergency123'(!}"script$
```

This worked as intended and the website gave us an alert shown in Figure 3.2.

3.1.1.2 Proposed solutions

We see that there is a simple output encoding of the input field which is a step toward the right direction. Encoding is a good short term solution but not sufficient for long term. We suggest that there wouldn't be any special characters allowed or that the backend code would simply remove any other characters than letters and numbers.

If it is intended that users are able to input special characters, the next best solu-

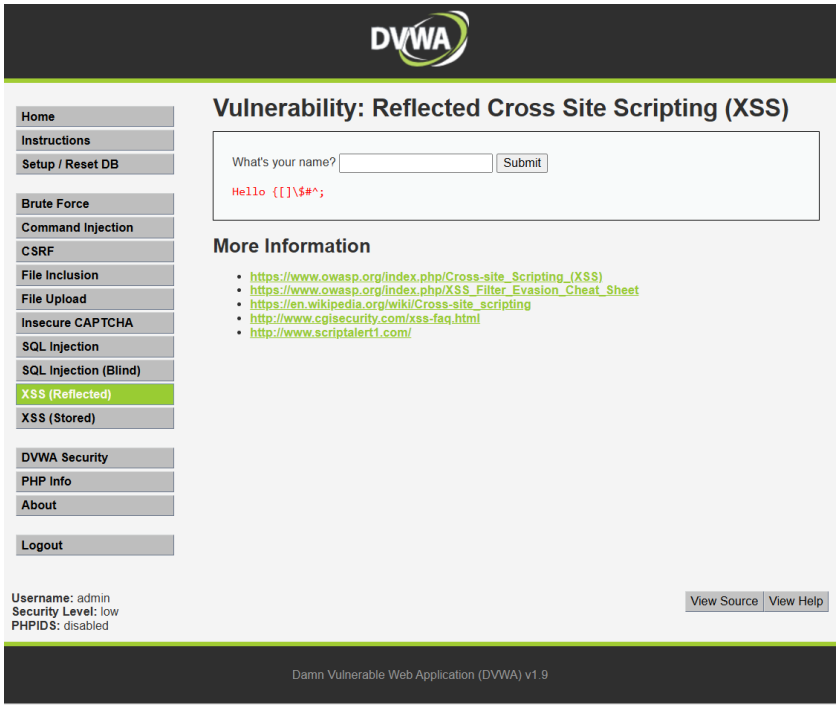


Figure 3.1: DVWA XSS input field



Figure 3.2: Alert from the DVWA website

tion would be to sanitize the text from the input field. This way, even with the special characters, the text would be non-executable.

3.1.2 Command Execution (Injection)

Command execution/injection happens when a user can input arbitrary commands into a vulnerable application which are executed on the host operating system. The commands are executed usually with the privileges that the vulnerable application has. For example, if the application has root level priveleges, the user can also execute commands at the root level. For our penetration test, we did not need root level privileges. We were testing only directory traversal and finding and reading a specific file.

Basic information about this issue is presented in Table 3.2.

Description	DVWA web application's host device executed our arbitrary commands from the input field. We were able to traverse directories and locate a specific file.
CVSS Base Score	9.6
Exploitability	High
Business impact	The privileges are unknown but if the application does not have root access, the attacker can only read all the files that are not protected by required root access. The attacker will be able to gather information about the computer for possible future attacks. If the user has root access, data confidentiality, integrity and availability are all highly compromised meaning that all information on the host machine could be read, changed or deleted and the attacker could do exploit this for example by installing ransomware and demanding money from the business.
References to classifications	OWASP
	CWE-78: Improper Neutralization of Special Elements used in an OS Command
Affected input	The user-controlled input field in the DVWA page.
Affected output	• The directories of the host machine were traversed. • The files inside the directories were read.

Table 3.2: Issue #2: Command excecution (injection) vulnerability on the DVWA website

3.1.2.1 Minimal proof of concept

There is an input field for the user to input an IP address on the command injection subsite. The problem here is that we can use the semicolon ";" symbol to start a new command which can be anything we want. We needed to find a specific file on the host system so we injected the input field with the following command that would find a file with the name "secret" anywhere in the host system:

```
1.1.1.1; find / -type f -name "secret*"
```

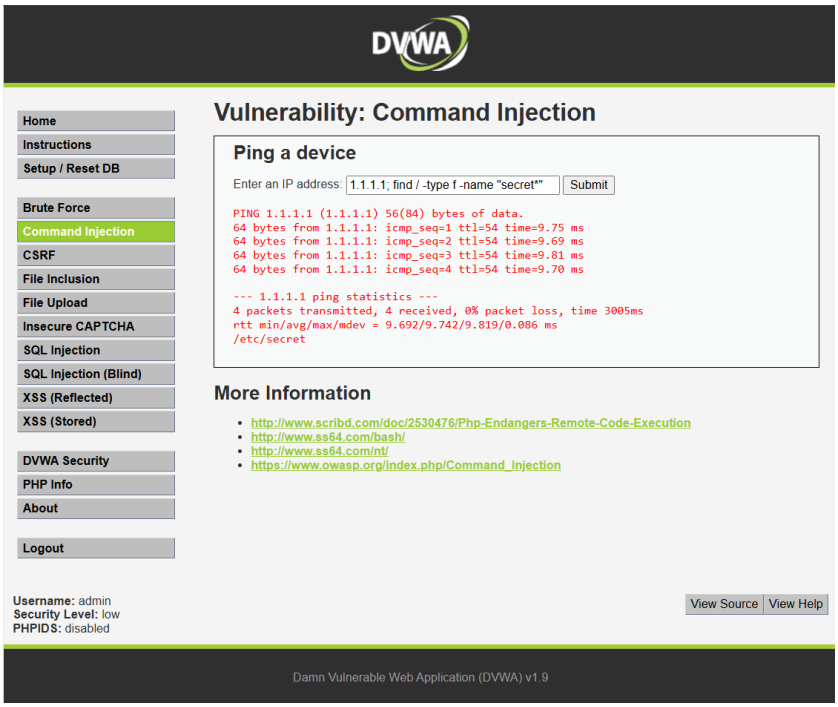


Figure 3.3: Command injection input field with our command

As seen in Figure 3.3, the website shows us a path to the file we're looking for:
`/etc/secret`
We read the file with the "cat" command:

```
1.1.1.1; cat ../../../etc/secret
```

Which gives us the string show in Figure 3.4.

```
ZjJmZmZlMjU1NWM4Nzk1M2RmZjU0NTM2N2Q5ZGRjZTU5MmNjMzBjMA==
```

3.1.2.2 Proposed solutions

For the short term, it is advised to completely disable this input field until the vulnerability is fully patched.

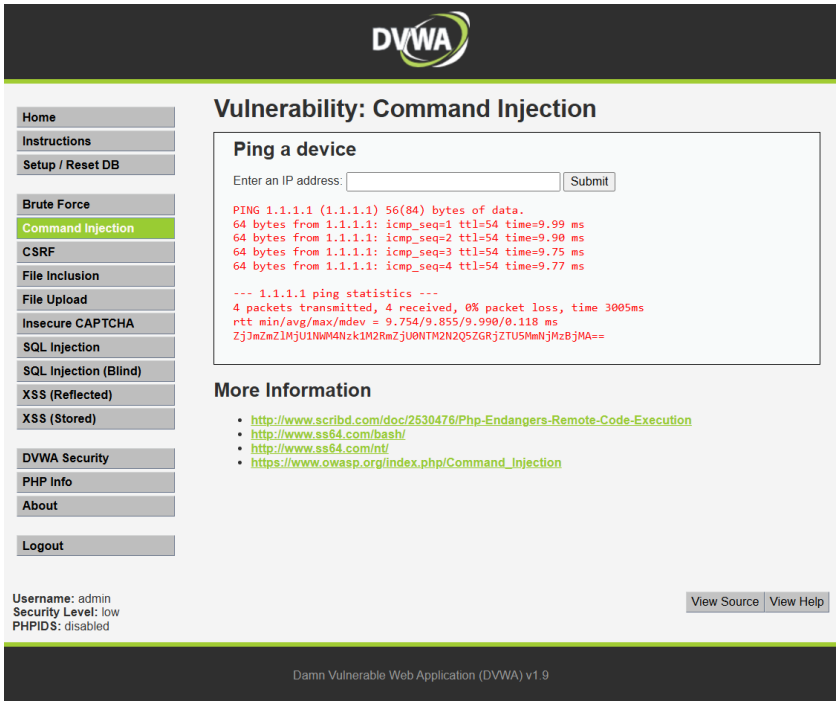


Figure 3.4: The fetched string on the last line of the output

Assuming that this functionality is wanted from the web application, we suggest that the backend code would not execute any other text than pinging the IP address, essentially disabling or deleting any other text after the IP address.

3.1.3 File Upload

In a file upload vulnerability, the attacker is able to upload a potentially malicious file to the server and the goal for the attacker is to usually execute that file which contains malicious code. The harm that this file upload can do depends on the type of software but in our case we were able to scan directories and read files with the payload we created.

Basic information about this issue is presented in Table 3.3.

Description	A malicious payload file was able to scan directories and read files within those directories.
CVSS Base Score	9.6
Exploitability	High
Business impact	An unauthorized user is able to read files, so confidentiality is compromised. The threat actor could potentially upload a file that deletes or changes directories or files alike meaning that also integrity and availability are compromised.
References to classifications	OWASP
	CWE-434:Unrestricted Upload of File with Dangerous Type
Affected input	A button through which the user can select a file and a button to upload it.
Affected output	• The directories of the host machine were scanned. • The files inside the directories were read.

Table 3.3: Issue #3: A critical file upload vulnerability on the DVWA website.

3.1.3.1 Minimal proof of concept

When entering the DVWA "File Upload" subsite, we are able to choose and upload a file shown in Figure 3.5.

We made the following .php filetype payload to scan the previous directory:

```
<?php
print_r(scandir(".."))
?>
```

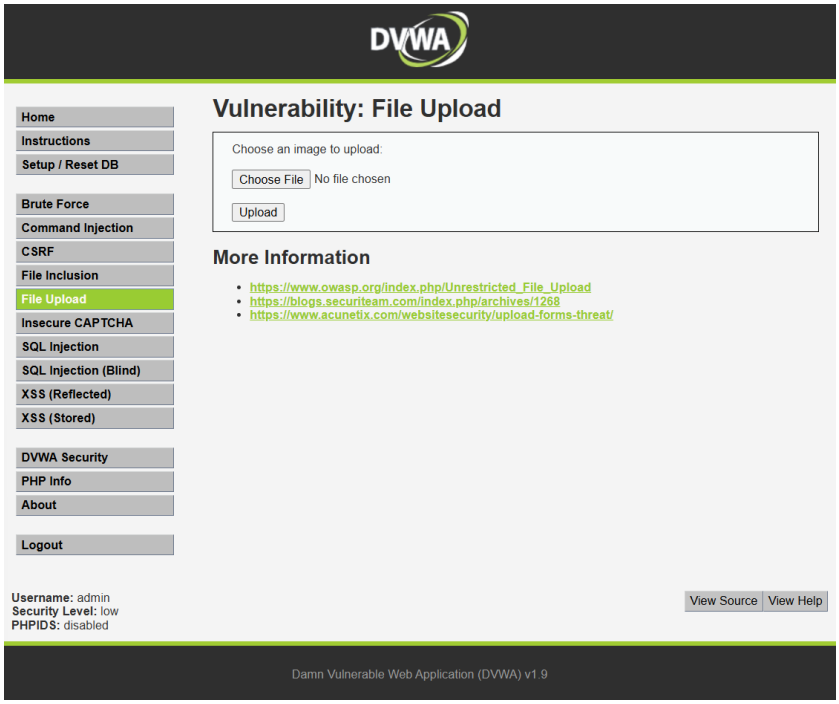


Figure 3.5: File upload page on the DVWA website

When we upload it to the server, we get the response indicated in Figure 3.6.

```
../../hackable/uploads/malicious-php-payload.php succesfully uploaded!
```

Figure 3.6: Response to our .php payload from the DVWA website

Now that we know the file path where the file was uploaded, we can open our file on the browser. When we open "http://193.167.189.112:2907/hackable/uploads/malicious-php-payload.php" (the IP and port were the website's hostname during our penetration test), we see the following output:

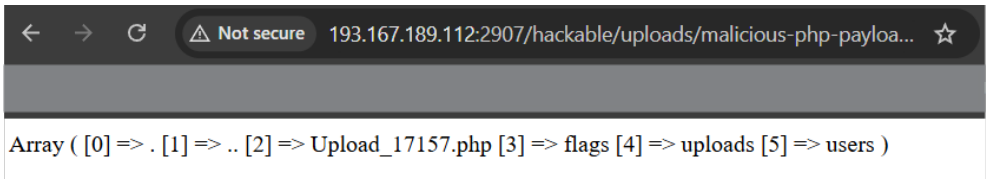


Figure 3.7: The output of our uploaded payload file

This shows us that there are 3 directories and 1 file called "Upload_17157.php" in the "uploads" directory.

We want to read the .php file so we make another simple payload:

```
<?php
readfile( "../Upload_17157.php" );
?>
```

Uploading this to the site, opening it and inspecting the blank site with our browser's developer tools shows us the contents of the file.

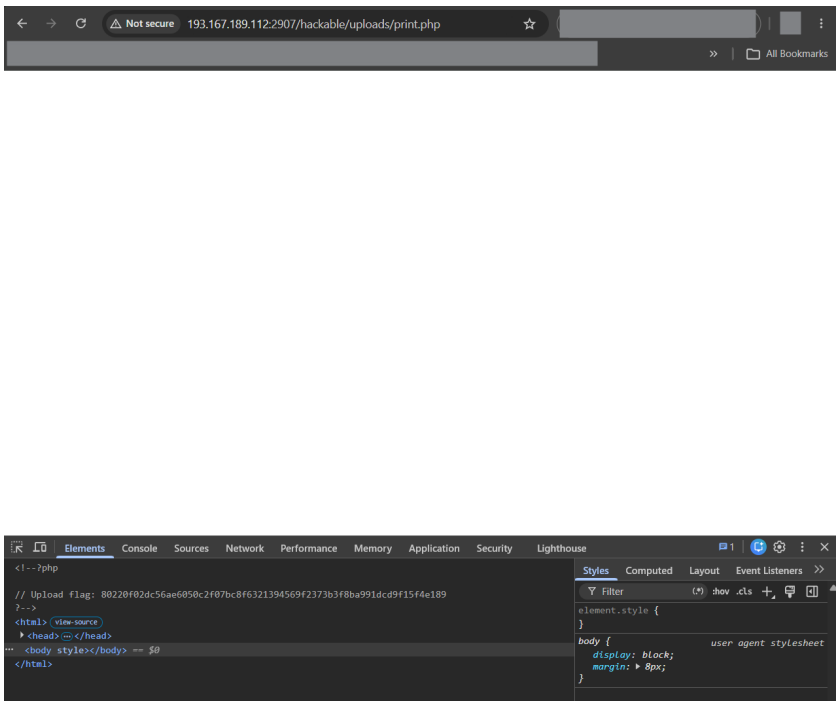


Figure 3.8: The contents of the .php file shown with developer tools

3.1.3.2 Proposed solutions

With file uploads are something to be very careful with and it is utmost importance for the application to never accept a file without inspecting it first and sanitazing or completely removing it.

The short term solution for DVWA would be to accept only the image files like ".png" and ".jpeg" but there could still be malicious metadata within the image that can overwrite or change files on the host system.

The long term solution would include accepting only image files (preferrably not too many image files so it is easier to manage security concerns) and inspecting the file for any malicious metadata/code which then would be either removed or sanitized.

3.1.4 File Inclusion

The file inclusion vulnerability allows an attacker to read contents of files they were not supposed to have access to. It could also lead to code execution on the web server, denial of service (DoS) or access to sensitive information depending on the severity. In our penetration test, we exploited this vulnerability by traversing the DVWA directory and reading the contents of a specific file.

Basic information about this issue is presented in Table 3.4.

Description	A file not meant to be accessed was read by using a payload in the url of the website.
CVSS Base Score	6.5
Exploitability	High
Business impact	An unauthorized user may be able to read any file within DVWA. This could expose sensitive information leading to a potential high confidentiality issue.
References to classifications	OWASP
	CWE-98: Improper Control of Filename for Include/Require Statement in PHP Program ('PHP Remote File Inclusion')
Affected input	The file parameter in the DVWA URL.
Affected output	The contents of the targeted file were returned by the web application.

Table 3.4: Issue #4: File inclusion vulnerability where a targeted file's contents were read through the DVWA URL.

3.1.4.1 Minimal proof of concept

Upon entering the file inclusion site, we see three files which are not of interest to us (shown in Figure 3.9).

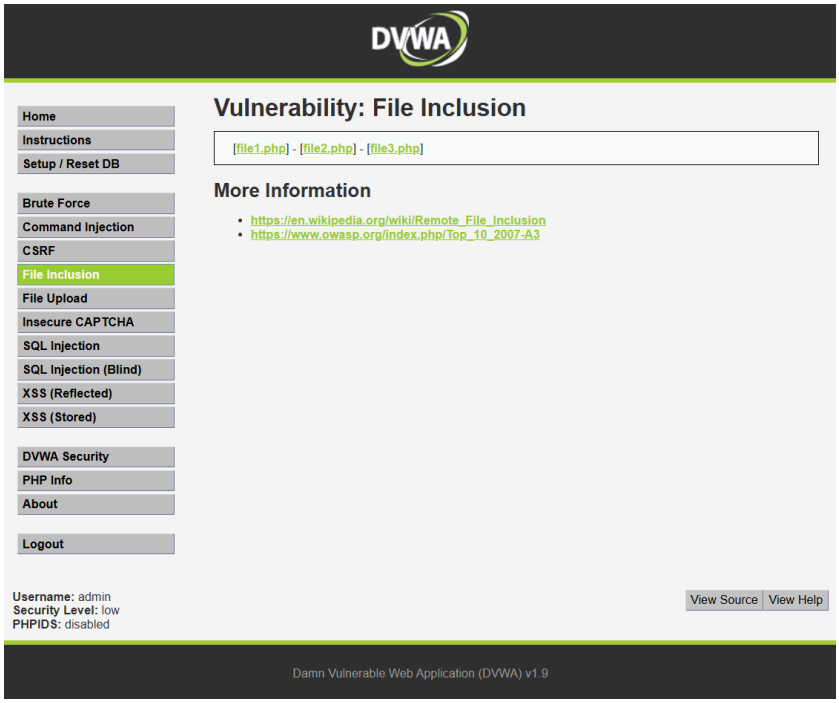


Figure 3.9: The file inclusion site on DVWA

What we focused on in our penetration test was the URL that had a specific syntax (?page=include.php):

`http://193.167.189.112:2905/vulnerabilities/fi/?page=include.php`

When inspecting the source code, we can see that the PHP $_GET function is structured so that there is no filtering meaning that the PHP-code is not validating the user input (Figure 3.10). This means that we are able to traverse the directories and possibly scan and read any file.



Figure 3.10: File inclusion site source code (no user input validation)

We knew already that the flag will be located at `/var/www/dvwa/hackable/flags/` and the file inclusion site's URL was `/var/www/dvwa/vulnerabilities/fi/?page=include.php` which meant that we only needed to make a payload relative to the file inclusion site location that would read the contents of the specific file.

To know which filename we need to read the contents of, we checked the index of `/hackable/flags` (Figure 3.11):



Figure 3.11: Index of `/hackable/flags`

We see that there is a file called 24407.php. So to read the contents of that file, we made the following payload:

`../../hackable/flags/24407.php`.

The content of the file is shown in Figure 3.12:

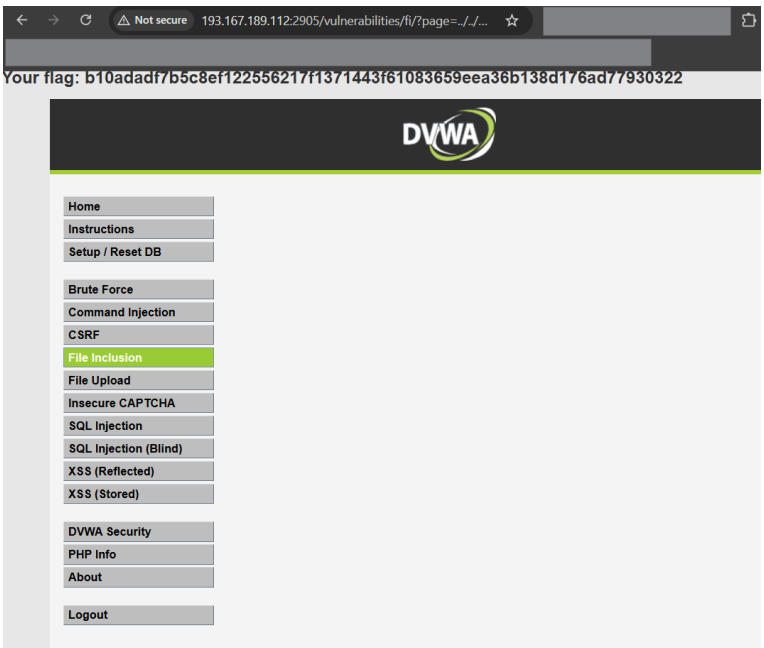


Figure 3.12: Contents of the 24407.php file

3.1.4.2 Proposed solutions

For both the short and long term we suggest as a solution to disable the user being able to submit any input to the URL / framework API. But if that is not possible or the feature is wanted, it is best to have a filter allowing only selected files to be viewed.

3.1.5 SQL Injection Without Randomization

SQL injection means that there is an input box (e.g. a login page with input) where an attacker can input an SQL query in order to fetch data from the backend of the web application. "Without randomization" in this case describes that we know the target prehand on top of getting output from the DVWA which will help us craft our attack. In our penetration test, we did a UNION-based SQL injection attack meaning we used the UNION statement in the SQL query in order to get our desired result.

Basic information about this issue is presented in Table 3.5.

Description	An SQL query gave us unauthorized access to user information from the website application's database.
CVSS Base Score	7.5
Exploitability	High
Business impact	Unauthorized users are able to read data from the database where they can see the user's email and possibly more information. It could be possible for the threat actor to see all the information stored on the database. Therefore confidentiality is compromised.
References to classifications	OWASP
	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') (4.20)
Affected input	Input field for user ID.
Affected output	User information from the web application's database was shown on the website.

Table 3.5: Issue #5: SQL injection vulnerability in an input field on the DVWA

3.1.5.1 Minimal proof of concept

When we enter the SQL injection site on DVWA, we see an input box where we can write a user ID (FIgure 3.13):

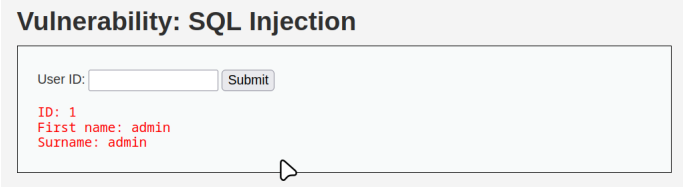


Figure 3.13: DVWA SQL Injection site input box

To find an email address from another table, we had to check what tables the database has by using this command:

```
ÚNION SELECT table_name, table_schema FROM information_schema.tables #
```

Doing that command shows us a ton of different tables but the most relevant one for us is called "vips". So in order to fetch an email from this table, we crafted the following injection:

```
únion select id, email from dvwa.vips #
```

And with this, we got an email:



Figure 3.14: Email fetched successfully from a SQL database.

3.1.5.2 Proposed solutions

The best and the long term solution for this issue would be to implement prepared statements and input validation together. The answer for the short term solution would be the same since there is hardly a simpler way to fix this issue.

3.1.6 Brute-Force Login

A brute-force attack attempts to gain unauthorized access by systematically trying different username and password combinations. During our penetration test, we used Hydra to automatically test credentials against the DVWA brute-force login page. A valid password was discovered, allowing us to successfully log in to the application.

Basic information about this issue is presented in Table 3.6.

Description	Description goes here.
CVSS Base Score	10.0
Exploitability	High
Business impact	An attacker could gain unauthorized access to a user account by repeatedly attempting different password combinations. Depending on the privileges of the compromised account, this could allow the attacker to access sensitive information or perform unauthorized actions within the application.
References to classifications	OWASP
	CWE-307: Improper Restriction of Excessive Authentication Attempts
Affected input	The username and password fields on the DVWA brute-force login page.
Affected output	A valid username and password combination was discovered, allowing successful authentication to the DVWA application.

Table 3.6: Issue #6: Brute-force vulnerability on the DVWA login page

3.1.6.1 Minimal proof of concept

To start, we needed to get our PHP session ID using developer tools after opening the website (Figure 3.15):

Then we recorded our status info about our credentials and IP address, and made an SSH connection to an attacker machine which is shown in Figure 3.16:

By using the command `sudo ifconfig` on the Linux machine terminal emulator, we discovered that our IP address is "173.3.106.2".

We had information that this attacker machine was in the same network as the computer which hosts the DVWA so we could scan it for open ports and running services (Figure 3.17):

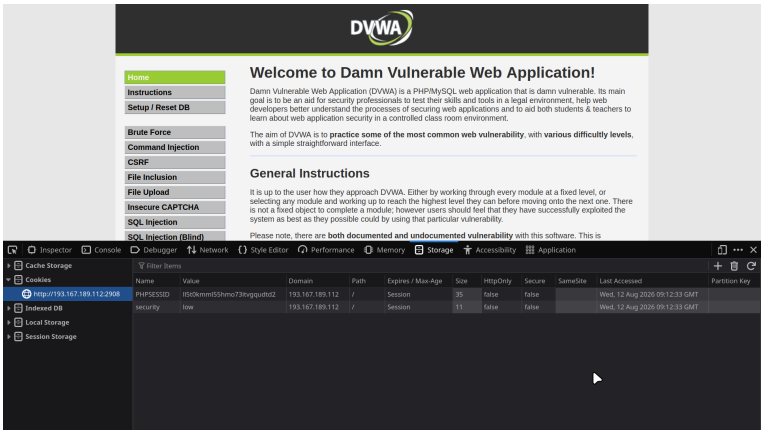


Figure 3.15: We had to record our PHP session ID

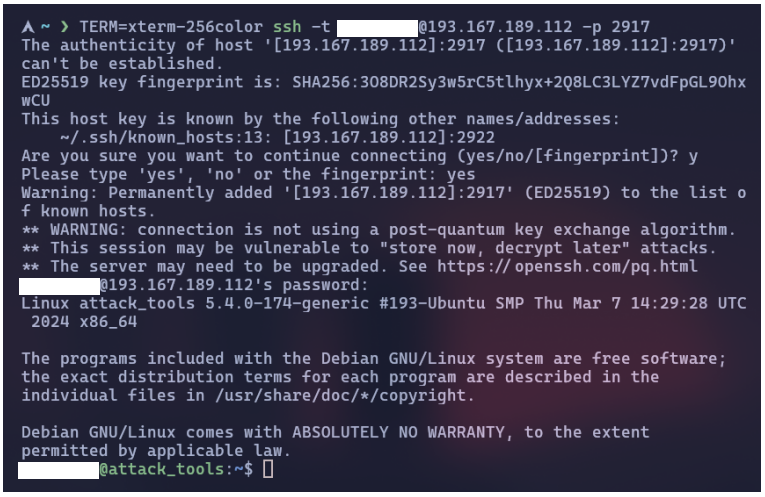


Figure 3.16: SSH connection to an attacker machine.

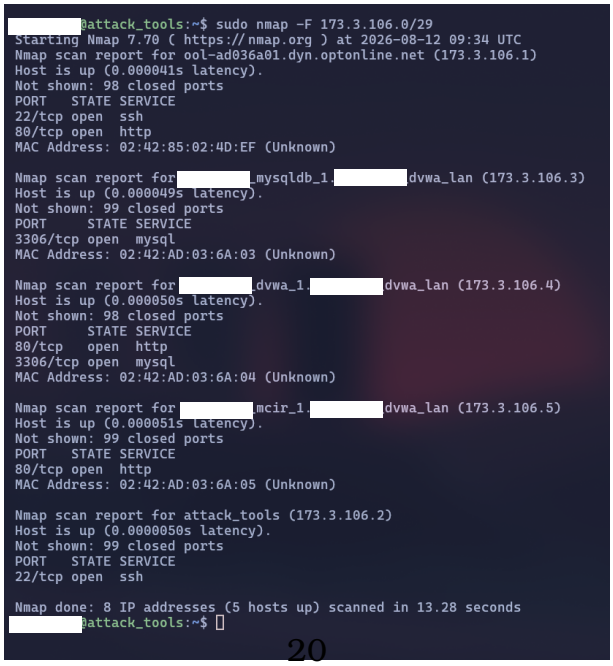


Figure 3.17: NMAP scan of the network that DVWA's host machine is in.

Next, with all the information available to us and by using this command:

```
hydra 173.3.106.4 -F -V -L /usr/share/usernames -P /usr/share/common.hydra \
173.3.106.4 -F -V -L /usr/share/usernames -P /usr/share/common.txt http-get-form \
"/vulnerabilities/brute/:username=^UStxt \
http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=
Login:F=Username and/or password incorrect.:H=Cookie\: \
PHPSESSID=ll5t0kmml55hmo73itvgqudtd2; security=low''
```

We were able to brute force the login information for someone's account called "paul":



Figure 3.18: Successful brute force login

Afterwards we continued on the DVWA website and input the login information getting the following message:

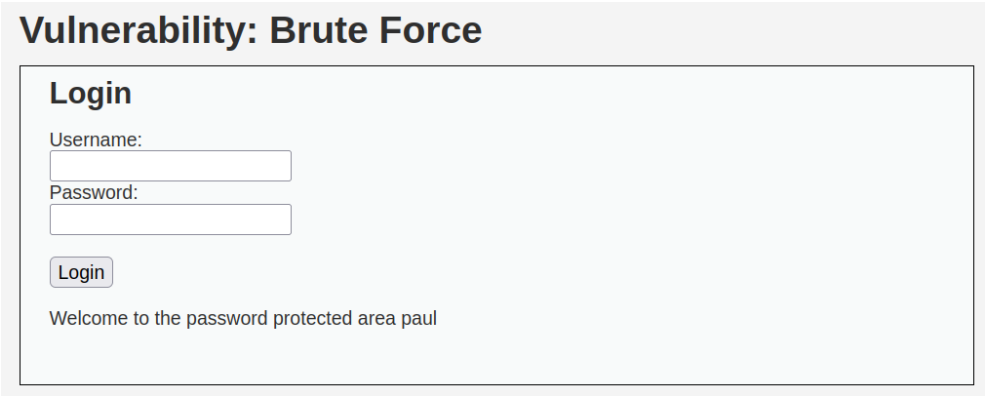


Figure 3.19: Successful login message

3.1.6.2 Proposed solutions

As a short-term solution, the application should limit the number of authentication attempts that can be made within a certain period of time. This can be implemented through rate limiting or temporary account lockouts after a number of failed login attempts.

A longer-term solution would be to implement several layers of protection. Strong password requirements should be used, while multi-factor authentication (MFA) could provide additional protection even if a password is successfully guessed. The application should also monitor and log repeated failed authentication attempts so that potential brute-force attacks can be detected and investigated.

4 Appendices

4.1 Network Scan Results

The following appendix contains the results of the network scan performed against the `173.3.106.0/29` network range. The scan was conducted in order to identify active hosts, open ports, and exposed services within the network.

Table 4.1 contains the results of the network scan for each open host.

IP Address	Hostname	Open Ports	Exposed Services	Ports Not Shown
173.3.106.1	`ool-ad036a01.dyn.optonline.net`	22/tcp, 80/tcp	SSH, HTTP	98
173.3.106.2	`attack_tools`	22/tcp	SSH	99
173.3.106.3	`_______mysqldb_1._____dvwa_lan`	3306/tcp	MySQL	99
173.3.106.4	`__________dvwa_1._____dvwa_lan`	80/tcp, 3306/tcp	HTTP, MySQL	98
173.3.106.5	`__________mcir_1._____dvwa_lan`	80/tcp	HTTP	99

Table 4.1: Results for the network scan

4.2 Reference and tool links

OWASP Website - Official OWASP website
CWE Website - Official CWE (Common Weakness Enumeration) website
CVSS Calculator - Website for calculating the CVSS scores for each vulnerability
DVWA Github Website - Official Github repository for the DVWA (Damn Vulnerable Web Application)